

# A10 — Unity Partition

This is the final architectural pass before we move from planning into **actual Unity construction**.

The purpose of A10 is **not to divide Phantom Q back into disconnected pieces**. The Headquarters remains one canonical building. We are deciding how Unity should internally organise that building so we can develop, test and optimise it without breaking the whole system.

The rule I would lock is:

**One world to the player; multiple controlled partitions to Unity.**

## A10.1 — Three different things must stay separate

Unity needs three architectural layers:

```
PHANTOM Q
|
├─ WORLD
|   What physically exists
|
├─ SYSTEMS
|   What makes the world behave
|
└─ DATA / STATE
    What is currently true
```

For example, the Red Team door is:

```
WORLD
Door model + reader + collider + animation

SYSTEM
Access / clearance service

STATE
PQ-004 mission access = TRUE
```

We should **never put all three responsibilities into one** `RedTeamDoor.cs` **script**.

That separation will matter enormously as the project grows.

## A10.2 — Runtime scene architecture

Eventually I would structure the running HQ like this:

```
00_BOOT
+
10_HQ_SHELL
+
20_OPERATIONAL_CORE
+
30_SPECIALIST_WING
+
40_SECURITY_WING
+
50_TECHNICAL_WING
+
60_SERVICE_INFRASTRUCTURE
```

All are loaded into **one world coordinate system**.

The learner never sees a loading screen saying:

| Loading SOC...

They simply walk through the same Headquarters.

---

## A10.3 — 00\_BOOT — persistent systems

This scene contains **no Headquarters geometry**.

It contains the systems that should survive regardless of which world content is loaded.

```
00_BOOT
|
|— ApplicationRoot
|
|— GameStateService
|
|— CampaignService
|
|— PlayerProfileService
|
|— AccessService
|
|— InteractionService
|
|— SaveService
```

```
|
|— SceneStreamingService
|
|— AudioService
|
|— InputService
|
|— BackendService
|
|— QKDSERVICE
|— DebugService
```

This becomes the application foundation.

We should avoid a situation where:

```
CentralOperations.unity
contains GameManager

SOC.unity
contains another GameManager
```

There is only one runtime authority for each system.

---

## A10.4 — 10\_HQ\_SHELL — the canonical building

This is one of the most important partitions.

It owns the **architecture that must line up perfectly across the whole HQ**:

```
10_HQ_SHELL
|
|— Main Floors
|— Structural Walls
|— Glass Partitions
|— Corridors
|— Security Vestibules
|— Door Frames
|— Secure Doors
|— Access Readers
|— Main Signage
|— Player Navigation Geometry
|— Cutaway Wall Sections
|— Room Boundaries
```

The shell is always loaded while inside Headquarters.

This is important because we do not want:

```
SOC scene owns one half of corridor
Central Ops scene owns other half
```

and later the walls stop lining up.

**Structural architecture belongs to the HQ shell.**

---

## A10.5 — Doors belong to the shell

I would also put the important security boundaries into the shell:

```
T0 - HQ Entry Gate
T1 - SOC portals
T2 - Red Team vestibule
T3 - Technical Gate
T4 - Advanced Technical Gate
T5 - High-Trust Vestibule
```

Why?

Because they are **relationships between zones**, not content belonging exclusively to either side.

This prevents duplicated doors and conflicting access states.

---

## A10.6 — Operational Core partition

I would group:

```
20_OPERATIONAL_CORE
|
├─ Reception / Identity
└─ Central Operations
```

These are the two most persistent areas in the current campaign.

Room content includes:

```
Reception desks
identity station
staff markers
WS01-WS04
main operations display
Command/Authorisation sub-zone
```

```
room-specific lighting props
room audio emitters
interaction points
```

But the outer walls remain in `HQ_SHELL`.

---

## A10.7 — Specialist Wing partition

Group the heavily interconnected educational/communication spaces:

```
30_SPECIALIST_WING
|
|— Foundations
|— Cryptography Lab
|— Communications
```

This grouping makes technical sense because A5 confirmed that these three form a frequent operational cluster.

Especially:

```
CRYPTO
↔
COMMUNICATIONS
```

We don't gain much by constantly loading/unloading them separately.

---

## A10.8 — Security Wing partition

```
40_SECURITY_WING
|
|— SOC
|— Red Team Operations
```

They are operationally related but still physically separated by T2.

This partition owns things such as:

```
SOC case board
session consoles
trace table
correlation wall
analyst workstations
```

```
Red Team target table
authorisation console
attack-surface display
operational station
artefact search system
```

Again, T2 itself stays in the shell.

---

## A10.9 — Technical Wing partition

I would place:

```
50_TECHNICAL_WING
|
|— Engineering
|— Quantum Wing
|— Advanced Compute
|— Secure Core interior
```

These spaces are largely future-facing in the current cyber campaign, so grouping them gives us a huge practical advantage.

For early Release 1 builds we can have:

```
Technical Wing
LOW DETAIL / LOCKED
```

while still showing that it physically exists.

Later we upgrade the contents without rebuilding the HQ.

---

## A10.10 — Service infrastructure partition

Separate:

```
60_SERVICE_INFRASTRUCTURE
|
|— Service Entry
|— Technical Service Spine
|— Equipment Staging
|— Maintenance Markers
|— Utility/Back-of-House elements
```

This is not initially player-facing.

Keeping it separate means we can dramatically reduce its runtime detail in normal gameplay.

---

## A10.11 — Why not one Unity scene per room?

Because that would create unnecessary complexity.

We have eleven rooms, but several are constantly used together.

If every room were separate:

```
Load Crypto
Unload Foundations
Load Communications
Unload Crypto
Load Crypto again
...
```

we create technical work without improving the player's experience.

Our partitions follow **operational clusters**, not arbitrary room count.

That is the right level of granularity for this Headquarters.

---

## A10.12 — But each room remains independently controllable

This is important.

Although Crypto and Communications share one Unity scene, they still have separate roots:

```
30_SPECIALIST_WING
|
├─ ROOM_FOUNDATIONS
|
├─ ROOM_CRYPTO
└─ ROOM_COMMUNICATIONS
```

And each root has:

```
RoomController
RoomDefinition
RoomVolume
```

```
CameraProfile
AmbientProfile
InteractionRoot
MissionContentRoot
NPCMarkers
```

So we can still activate, debug and configure a room independently.

---

## A10.13 — Canonical Unity hierarchy

A loaded HQ might therefore look like:

```
PHANTOM_Q_RUNTIME
|
├── _SYSTEMS
|
├── _PLAYER
|
├── HQ_SHELL
|   ├── Structure
|   ├── Corridors
|   ├── Doors
|   ├── SecurityTransitions
|   └── RoomVolumes
|
├── OPERATIONAL_CORE
|   ├── Reception
|   └── CentralOperations
|
├── SPECIALIST_WING
|   ├── Foundations
|   ├── Crypto
|   └── Communications
|
├── SECURITY_WING
|   ├── SOC
|   └── RedTeam
|
├── TECHNICAL_WING
|   ├── Engineering
|   ├── Quantum
|   ├── AdvancedCompute
|   └── SecureCore
|
└── SERVICE_INFRASTRUCTURE
```

To the learner:



# one Headquarters

To Unity:

## controlled modules

---

### A10.14 — World coordinates are sacred

This is one of the rules I would be strict about.

We already established:

```
Central Operations  
centre = (0,0,0)
```

That remains true in **every Unity scene**.

For example:

```
SOC  
centre ≈ (+20, 0, +9)
```

Even while editing `40_SECURITY_WING.unity`, SOC should remain at:

```
(+20,0,+9)
```

Do **not** move it to:

```
(0,0,0)
```

just because we're editing SOC separately.

Otherwise additive scene alignment becomes fragile.

---

### A10.15 — Every partition root stays at world origin

Use:

```
OPERATIONAL_CORE_ROOT  
Position = (0,0,0)  
  
SPECIALIST_WING_ROOT  
Position = (0,0,0)
```

```
SECURITY_WING_ROOT  
Position = (0,0,0)
```

```
TECHNICAL_WING_ROOT  
Position = (0,0,0)
```

The **objects inside** are positioned according to canonical HQ coordinates.

This means additive loading lines everything up automatically.

---

## A10.16 — Room positions should never be hard-coded into gameplay

Bad:

```
C#  
  
if (player.position.x > 12 &&  
    player.position.x < 28)  
{  
    currentRoom = "SOC";  
}
```

Very fragile.

Instead each room has a proper:

```
RoomVolume
```

When the player enters:

```
RoomVolume  
↓  
RoomService  
↓  
CurrentRoom = SOC
```

Then the room can request:

```
SOC camera profile  
SOC ambient audio  
SOC room-entry UI  
SOC mission context
```

The architecture controls the state.

Not arbitrary coordinates buried in C#.

---

## A10.17 — World objects need stable IDs

Every important persistent object should have a unique canonical ID.

Examples:

```
HQ.WS.04

HQ.DOOR.T0.01

HQ.ROOM.SOC

HQ.ROOM.CRYPTO

HQ.SOC.TRACE.01

HQ.REDTEAM.AUTH.01

HQ.CRYPTO.SYM.01

HQ.CRYPTO.ASYM.01
```

The visible name can still be:

Workstation 04

But internally we need stable references.

This becomes crucial for saving state and multiplayer.

---

## A10.18 — Object ID must not depend on scene hierarchy

Do not identify things through:

```
GameObject.Find(
    "HQ/Operational/Core/Desk/Computer/WS04"
)
```

If we reorganise the hierarchy, everything breaks.

Instead:

```
PersistentObjectId  
= "HQ.WS.04"
```

Then systems reference the object's identity, not its current Unity path.

---

## A10.19 — Prefab partition

Anything reused should become a prefab.

### Architectural prefabs

```
PQ_Wall_1M  
PQ_Wall_2M  
PQ_Wall_4M  
  
PQ_GlassWall_2M  
PQ_GlassWall_4M  
  
PQ_Door_Standard  
PQ_Door_Secure  
PQ_Door_Double  
PQ_AccessReader
```

### Interaction prefabs

```
PQ_Workstation  
PQ_InteractionMarker  
PQ_EvidenceBoard  
PQ_TableInteraction  
PQ_Console  
PQ_RoomTrigger
```

### Character prefabs

```
PQ_Player  
PQ_CharacterBase  
PQ_Alice  
PQ_Bob_Remote  
PQ_Eve  
PQ_RedTeamOperator  
PQ_GenericStaff
```

## A10.20 — Prefab variants

Unity Prefab Variants will be useful.

For example:

```
PQ_Workstation
|
├─ Workstation_Standard
├─ Workstation_04
├─ SOC_Workstation
└─ RedTeam_Workstation
```

Shared behaviour stays inherited.

Individual identity and visuals vary.

Similarly:

```
PQ_SecureDoor
|
├─ SOC_Door
├─ RedTeam_Door
├─ Quantum_Door
└─ SecureCore_Door
```

We should not duplicate complete prefabs just because the signage differs.

---

## A10.21 — Script architecture should follow the same partitions

I'd establish these main C# namespaces/assemblies:

```
PhantomQ.Core
PhantomQ.World
PhantomQ.Characters
PhantomQ.Interaction
PhantomQ.Security
PhantomQ.Campaign
PhantomQ.Presentation
PhantomQ.Integrations
PhantomQ.Debug
```

This maps very closely to what we already designed.

## A10.22 — Use Assembly Definitions

This is worth doing early.

Unity assembly definition files let us control which code layers may reference which others.

For example:

```
PhantomQ.Core
    ↑
    |
PhantomQ.World
PhantomQ.Security
PhantomQ.Interaction
    ↑
    |
PhantomQ.Campaign
    ↑
    |
PhantomQ.Presentation
```

And:

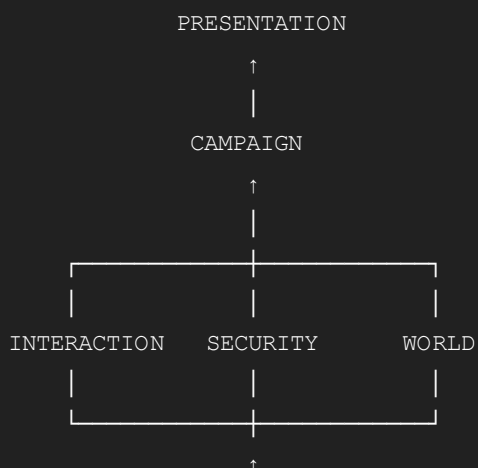
```
PhantomQ.Integrations
```

implements interfaces defined lower in the architecture.

This helps prevent circular dependencies.

## A10.23 — Dependency direction

I would lock this:



Integrations sit beside the system and implement service interfaces.

The important rule:

**Core never depends on Incident 06.**

And:

**Security never depends on a specific SOC UI.**

Higher-level systems use lower-level systems, not the reverse.

---

## A10.24 — Avoid a giant `GameManager.cs`

This is one of the easiest ways Unity projects become difficult to maintain.

We should not create:

```
GameManager.cs
```

```
8,000 lines:
```

```
player
```

```
Alice
```

```
doors
```

```
missions
```

```
QKD
```

```
UI
```

```
multiplayer
```

```
sound
```

```
everything
```

Instead:

```
CampaignService
```

```
AccessService
```

```
InteractionService
```

```
RoomService
```

```
CharacterService
```

```
SaveService
```

```
QKDSERVICE
```

```
NetworkService
```

Each has one bounded responsibility.

---

## A10.25 — Campaign code must not live in room scripts

For example, Crypto Lab should not contain:

```
C#  
  
if (Incident02 && scene == 4)  
{  
    CreateA7();  
}  
  
if (Incident05 && scene == 20)  
{  
    ShowSignature();  
}
```

That is exactly what we must avoid.

Instead:

```
CampaignService  
↓  
reads Incident/Scene definition  
↓  
requests world actions  
↓  
Crypto room activates relevant objects
```

The room is a capability.

The campaign decides when/how it is used.

---

## A10.26 — Data partition

We already have excellent candidates for ScriptableObjects.

```
DATA  
|  
├─ RoomDefinition  
├─ CharacterDefinition  
├─ AccessPolicy  
├─ CameraProfile  
├─ InteractionDefinition  
├─ CampaignDefinition  
├─ IncidentDefinition  
├─ ActDefinition  
└─ SceneDefinition
```



```
| ObjectiveDefinition
| EvidenceDefinition
| DialogueDefinition
```

These are **definitions**.

They should not be treated as mutable runtime save data.

---

## A10.27 — Definition vs state

This distinction must be explicit.

Example:

### Definition

```
RedTeamRoomDefinition

Name:
Red Team Operations

NormalAccess:
Restricted

CameraProfile:
RedTeam_Default
```

### Runtime state

```
RedTeamRoomState

CurrentAccess:
MISSION

Case:
PQ-004

Active:
TRUE
```

Definitions describe **what something is**.

State describes **what is currently happening**.

Do not mix them.

---

## A10.28 — Campaign state must survive scene loading

For example:

```
A7  
COMPROMISED
```

cannot be stored only inside the Crypto scene.

If Crypto unloads:

```
A7 state must not disappear.
```

Instead:

```
CampaignRuntimeState  
|  
├─ CurrentIncident  
├─ CurrentScene  
├─ Evidence  
├─ Capabilities  
├─ AccessGrants  
├─ SecurityState  
├─ KeyState  
└─ MissionState
```

exists outside the world partitions.

The world **reflects** state.

It does not own campaign truth.

---

## A10.29 — This is critical for multiplayer later

Eventually:

```
SERVER  
= authoritative truth
```

while Unity has:

```
CLIENT REPRESENTATION
```

So if we already architect the game around:

```
STATE
  separate from
  WORLD
```

then multiplayer becomes much easier.

Later:

```
LocalCampaignState
```

can become:

```
ServerBackedCampaignState
```

without redesigning every room.

---

## A10.30 — Integration partition

All external communication belongs behind interfaces.

For example:

```
IQKDService
IRoomNetworkService
IBackendService
IFileService
IChatService
```

Development implementations:

```
MockQKDService
MockBackendService
```

Production/local-network implementations:

```
FlaskQKDService
FlaskBackendService
```

The campaign code shouldn't care which one is active.

---

## A10.31 — Example

Incident system asks:

```
IQKDService.Measure(...)
```

During development:

```
MockQKDService  
returns simulated result
```

During hardware demonstration:

```
PhysicalQKDService  
↓  
Flask  
↓  
Raspberry Pi  
↓  
Arduino / physical system
```

Same Unity gameplay.

Different service implementation.

That's exactly what we want.

---

## A10.32 — Character partition

Character architecture should similarly separate:

```
CHARACTER  
|  
├─ Definition  
├─ Runtime State  
├─ Visual Model  
├─ Movement Controller  
├─ Animator  
├─ Interaction Behaviour  
└─ Mission Behaviour
```

Alice's model should be replaceable without touching:

```
Alice mission identity  
Alice permissions  
Alice dialogue  
Alice movement markers
```

This protects us while art quality improves.

---

## A10.33 — Character location state

We should not save:

```
Alice x = 2.78392  
z = -2.83721
```

after every scene unless truly necessary.

For campaign characters, often save semantic state:

```
Alice.Location  
= CENTRAL_OPS  
  
Alice.Marker  
= WS04_SOURCE
```

Then the world can spawn/place her correctly.

This is more robust against map adjustments.

---

## A10.34 — Marker system

Our rooms need reusable named markers.

Examples:

```
CentralOperations  
|  
├─ MARK_PlayerSpawn  
├─ MARK_AliceInternalEntry  
├─ MARK_Alice_WS04  
├─ MARK_Player_WS04  
├─ MARK_CommandStaff  
└─ MARK_SOC_Liaison
```

Crypto:

```
MARK_Alice_Source  
MARK_Player_SymmetricBench  
MARK_Player_AsymmetricBench
```

SOC:

```
MARK_Player_CaseBoard  
MARK_Player_TraceTable  
MARK_Eve_SecuritySupport  
MARK_RT_Liaison
```

Campaign sequences request markers by ID.

Not hard-coded coordinates.

---

## A10.35 — Camera partition

Presentation owns camera behaviour globally.

Rooms provide data:

```
Room  
↓  
CameraProfile
```

Interactions provide:

```
Interaction  
↓  
CameraAnchor / CameraMode
```

Then:

```
CameraSystem
```

decides how to blend.

No individual workstation should take direct ownership of `Camera.main`.

---

## A10.36 — Cutaway system belongs globally

The wall-cutaway controller belongs under:

```
PhantomQ.Presentation
```

while the wall objects simply declare:

CutawaySurface

That means we can change the cutaway algorithm once and the whole building benefits.

---

## A10.37 — Lighting partition

We should separate:

```
GLOBAL LIGHTING
+
ROOM LIGHTING
+
INCIDENT STATE
```

Example:

```
Global
day/ambient baseline

Central Ops
room-specific fixtures

Incident 04
subtle operational alert state
```

Campaign requests:

```
CentralOps.SetOperationalState(Restricted)
```

The room presentation decides appropriate lights/screens/audio.

The campaign should not manually change ten light intensities.

---

## A10.38 — Audio partition

Similarly:

```
GLOBAL AMBIENCE
HQ general

ROOM AMBIENCE
SOC / Crypto / Engineering etc.

INTERACTION AUDIO
```

```
door reader / terminal / workstation
```

```
INCIDENT AUDIO
```

```
alerts / state transitions
```

One AudioService coordinates priority and mixing.

---

## A10.39 — Mission content root per room

Each room should contain something like:

```
ROOM_SOC
|
|— PermanentContent
|
|— AmbientContent
|
|— MissionContent
|   |— Incident04
|   |— Incident05
|   |— Incident06
|— DebugContent
```

But do not necessarily activate all MissionContent objects simultaneously.

The CampaignService decides which set is relevant.

---

## A10.40 — We should avoid duplicating incident objects

Example:

Do not create:

```
SOC_Incident04_TraceTable
SOC_Incident06_TraceTable
```

if the physical trace table is the same equipment.

Instead:

```
SOC_TraceTable
```

is permanent.

Its data/state changes.



This will make the environment feel persistent.

---

## A10.41 — Evidence should be data-driven too

For example:

```
EvidenceDefinition

ID:
PQ004.NETWORK.172_18_44_7

Display:
172.18.44.7

Type:
NetworkLead

DiscoveryScene:
Incident04.Scene16

Persistent:
TRUE
```

Then the SOC evidence board reads discovered evidence from state.

We don't manually place every evidence card in every scene.

---

## A10.42 — Navigation partition

For our initial build, I would use:

### one HQ NavMesh

because the building is compact.

`HQ_SHELL` owns the primary navigation surface.

This avoids seams between rooms.

---

## A10.43 — Door permissions and navigation

NPC logic must check permission **before requesting a destination beyond a secure boundary**.

Example:

```
Alice wants Red Team
↓
AccessService
DENIED
↓
navigation request never issued
```

rather than letting Alice continuously walk into a locked door.

The door controls physical passage.

AccessService controls whether the route is legitimate.

---

## A10.44 — Later optimisation

If the Headquarters becomes much larger, we can split navigation by wing:

```
Operational NavMesh
Security NavMesh
Technical NavMesh
```

with proper links.

But for Release 1:

**keep navigation simple until performance tells us otherwise.**

Do not optimise ourselves into complexity prematurely.

---

## A10.45 — Scene streaming strategy

There should be three levels.

### Level 1 — Development / greybox

Load:

```
BOOT
+
ENTIRE HQ GREYBOX
```

Everything simultaneously.

This is what we should build first.

---

## Level 2 — Normal desktop production

Potentially load:

```
BOOT
HQ_SHELL
Operational Core
Specialist Wing
Security Wing
```

continuously.

Technical Wing can load when approaching T3 if necessary.

Given the compact HQ, modern desktop hardware may handle almost everything at once.

---

## Level 3 — Mobile/Web optimisation

Use more selective loading:

```
CURRENT_CLUSTER
+
ADJACENT_CLUSTER
```

For example player in SOC:

|                  |                  |
|------------------|------------------|
| Operational Core | Loaded           |
| Specialist Wing  | Loaded/reduced   |
| Security Wing    | Full             |
| Technical Wing   | Unloaded/reduced |

The player never notices.

---

## A10.46 — Do not build streaming first

This is another thing I would lock.

Our first HQ prototype should **not** begin with sophisticated streaming.

First prove:

whole building works

Then profile.

Then optimise.

Otherwise we'll spend time debugging scene loading before we even know whether walking through Central Operations feels good.

---

## A10.47 — Quality settings partition

Presentation should later support:

LOW  
MEDIUM  
HIGH

but world logic must not depend on quality.

For example:

Red Team door exists  
on all quality levels.

Only:

shadow quality  
texture resolution  
reflection quality  
LOD distances  
effect density

change.

Never remove gameplay-critical objects to gain FPS.

---

## A10.48 — Debug architecture

I strongly recommend creating development tools from the beginning.

A simple Phantom Q debug panel could expose:

CURRENT ROOM  
Central Operations

INCIDENT  
Prologue

SCENE  
07

PLAYER ACCESS  
Trainee

TELEPORT  
Reception  
Central Ops  
Crypto  
SOC  
Red Team

GRANT CAPABILITY  
Foundations  
Crypto  
SOC  
Red Team

SET INCIDENT  
Prologue  
01  
02  
03  
04  
05  
06

This will save us enormous amounts of time.

---

## A10.49 — Why debug teleport is acceptable

It is a **development tool**, not player gameplay.

When testing Incident 06, we shouldn't have to replay the entire Prologue every time.

So development mode can jump directly to:

```
Incident06.Scene14
```

and create the necessary mock state.

That is professional workflow, not cheating.

---

## A10.50 — Testing partition

I'd create three test types.

```
EDIT MODE TESTS
logic/data

PLAY MODE TESTS
Unity behaviour

MANUAL QUALITY TESTS
handling/visuals
```

Examples:

### EditMode

```
RedTeamAccessRequiresMission()
A7CannotBecomeTrustedAfterCompromise()
```

### PlayMode

```
SecureDoorOpensForValidAccess()
RoomVolumeChangesCurrentRoom()
```

### Manual

```
Does Alice stop naturally at WS04?
Does camera feel good?
```

Not everything should be automated, but the rules that can be automated should be.

---

## A10.51 — Folder structure after A10

I would now settle on:

```
Assets/
├── PhantomQ/
│   |
│   ├── Core/
│   |
│   └──
```

- └─ World/
  - └─ Architecture/
  - └─ Rooms/
  - └─ Doors/
  - └─ Navigation/
  - └─ Prefabs/
- └─ Characters/
- └─ Interaction/
- └─ Security/
- └─ Campaign/
  - └─ Prologue/
  - └─ Incident01/
  - └─ Incident02/
  - └─ Incident03/
  - └─ Incident04/
  - └─ Incident05/
  - └─ Incident06/
- └─ Presentation/
  - └─ Cameras/
  - └─ Lighting/
  - └─ Audio/
  - └─ Timeline/
  - └─ VFX/
- └─ UI/
- └─ Data/
- └─ Integrations/
  - └─ Mock/
  - └─ Backend/
  - └─ Multiplayer/
  - └─ QKD/
- └─ Art/
- └─ Scenes/
  - └─ 00\_BOOT.unity
  - └─ 10\_HQ\_SHELL.unity
  - └─ 20\_OPERATIONAL\_CORE.unity
  - └─ 30\_SPECIALIST\_WING.unity
  - └─ 40\_SECURITY\_WING.unity
  - └─ 50\_TECHNICAL\_WING.unity
  - └─ 60\_SERVICE\_INFRASTRUCTURE.unity
- └─ Dev/
  - └─ Debug/

└─ Prototypes/  
└─ Tests/

That is now specific enough to implement.

## A10.52 — What the first Unity build should actually use

This is important because we shouldn't immediately create all seven production scenes.

For the first build:

```
00_BOOT  
+  
01_HQ_MASTER_GREYBOX
```

That's it.

`01_HQ_MASTER_GREYBOX` contains the entire A4 floor plan:

```
Reception  
Central Ops  
Foundations  
Crypto  
Communications  
SOC  
Red Team  
Engineering  
Quantum  
Advanced Compute  
Secure Core  
service spine
```

all as ProBuilder/basic geometry.

Why?

Because before partitioning the final environment we need to verify:

```
whole-building scale  
walking distance  
camera  
sightlines  
NPC routes  
clearance boundaries  
room dimensions
```

as one physical system.



## A10.53 — Then partition only after greybox lock

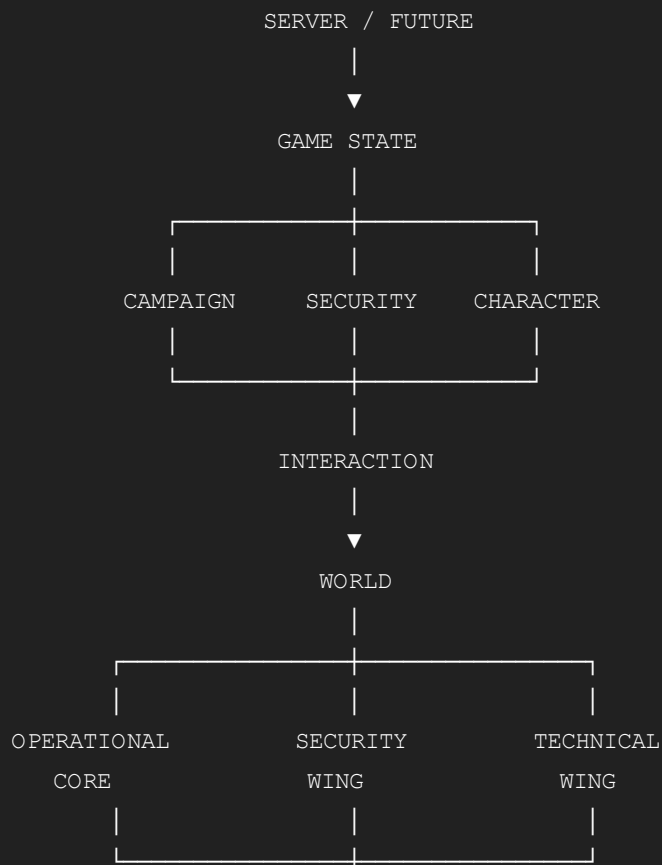
Development sequence:

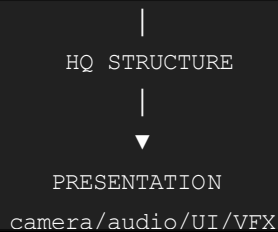
```
A1-A10 DESIGN
    ↓
FULL HQ MASTER GREYBOX
    ↓
WALK / TEST EVERYTHING
    ↓
MAKE FINAL SPATIAL CORRECTIONS
    ↓
HQ GREYBOX LOCK
    ↓
THEN SPLIT INTO
production partitions
```

This directly honours what you were saying earlier about **designing the whole first and using sections to control construction**.

## A10.54 — The full architecture from campaign to world

At this point our system is:





The player experiences that as:

## Phantom Q Headquarters

not as software modules.

---

### A10.55 — A1–A9 final loop

A10 does not require us to redesign the building.

#### A1 — Meaning

Still one persistent operational engineering/security organisation.

#### A2 — Hierarchy

Unity partitions preserve the spatial-security hierarchy.

#### A3 — Adjacency

Structural shell preserves the direct connections and security transitions.

#### A4 — Floor plan

Canonical world coordinates preserve the entire agreed layout.

#### A5 — Campaign

Campaign state is independent of scenes, so incidents can reuse the same world correctly.

#### A6 — Characters

Characters can navigate across partitions because world/navigation remains continuous.

#### A7 — Clearance

Doors and zone access are global systems rather than room-specific hacks.

#### A8 — Interaction

Rooms own reusable interaction capability; campaign activates the relevant state.

#### A9 — Camera/handling

Camera is global and state-driven, so crossing Unity partitions doesn't change control language.

**The architecture survives the loop.**

---

## A10 — What I would now lock

The production architecture should be:

ONE CANONICAL HQ

STRUCTURE

always coherent

ROOMS

grouped into operational partitions

SYSTEMS

persistent and independent

CAMPAIGN

data-driven

STATE

separate from world objects

CHARACTERS

reusable and role-driven

ACCESS

central service

INTERACTION

one common framework

CAMERA

one global state system

BACKEND

behind interfaces

UNITY SCENES

implementation partitions,

not separate game levels

And our **first physical Unity deliverable** is now unambiguous:

## Build 01 — Full Phantom Q HQ Master Greybox

Not Central Ops alone.

Not one polished room.

Not Incident 01.

We construct the **entire A4 Headquarters footprint at 1:1 scale in a single greybox scene**, including every room, corridor, security threshold and service spine. Then we put in the player and camera and run the A5/A6/A7/A9 tests against the actual geometry.

Only once that succeeds do we split it into the production Unity partitions above.

That completes **Phase A: Design the Whole System**. The next phase is no longer conceptual architecture—we can move into **Phase B and map the exact Unity greybox build sequence**, starting with creating the Unity 6 project correctly and constructing the full HQ shell from the coordinates we have established.